R codes for the NutNet-Stability project

Step2: Quantify stability components

Qiang Yang

2026-06-22

Overview

This document quantifies the multidimensional stability metrics used throughout the NutNet stability analyses. Starting from the cleaned analytical dataset (`df.tidy`) generated in Step 1, the workflow calculates functional and compositional dimensions of ecological stability for each nutrient-addition plot relative to an appropriate control plot within the same experimental block.

Specifically, the script calculates (i) temporal functional variability, (ii) temporal compositional variability, (iii) extent of functional change, and (iv) extent of compositional change. Stability metrics are calculated sequentially through time using observations accumulated from treatment year 1 onwards, thereby allowing the effects of treatment duration on stability to be evaluated.

The resulting datasets constitute the core analytical inputs for all subsequent statistical analyses, including mixed-effects models, dimensionality analyses, and figure generation. The code is extensively annotated to facilitate transparency, understanding, and reproducibility.

Output

Running this workflow generates the following outputs:

- `df.stability`: a plot-level dataset containing temporal functional variability, temporal compositional variability, functional resistance, and compositional resistance calculated for each plot and treatment year.
- A distribution figure summarizing the empirical distributions of the multidimensional stability metrics (not included in the manuscript).
- A correlation figure illustrating the pairwise relationships among the different dimensions of ecological stability (Fig. S9).

The object `df.stability` serves as the principal input for all subsequent statistical analyses and dimensionality calculations presented in the manuscript.

Setup work environment

```
> rm(list = ls())
> gc()
```

```
##          used (Mb) gc trigger (Mb) max used (Mb)
## Ncells 496512 26.6    1075400 57.5   686411 36.7
## Vcells 928342  7.1    8388608 64.0  1876578 14.4
```

```
> # necessary R packages
> Packages <- c("tidyverse", "magrittr", "vegan", "dlookr")
> pacman::p_load(char = Packages)
> # functions with conflicting status (presence in different packages used here)
> select <- dplyr::select
> summarise <- dplyr::summarise
> # working path
> main.path <- getwd()
> data.path <- paste0(main.path, "/mydata")
> set.seed(1234)
```

Section 1: Quantify stability components using biomass data

This include the functional variability (detrended and non-detrended) and functional resistance.

1.1 temporal functional variability

Temporal functional variability was quantified as: CV.treatment / CV.control using total biomass.

There might be multiple control plots in one block. First, select one control for each block.

```
> load(file = paste0(data.path, "/df.tidy.Rdata"))
> df.tidy.mass <- df.tidy %>% #only the plots with biomass data
+   filter(data.availability != "only coverage data") %>%
+   select(site_code:trt,biomass.data,total.mass) %>%
+   rename(data = biomass.data)
> df.controls <- df.tidy.mass %>%
+   unnest(cols=data) %>%
+   ungroup() %>%
+   filter(trt == "Control") %>%
+   select(site_code, block, plot, year_trt) %>%
+   distinct()
> set.seed(1234)
> df.controls.selected <- df.controls %>% # select the plot with largest year_trt
+   arrange(site_code, block, desc(year_trt)) %>%
+   group_by(site_code, block) %>%
+   slice_head(n = 1) %>%
+   ungroup() %>%
+   select(-year_trt)
```

Then calculate the coefficient of variation in each plot, both originally and detrendedly.

```
> df.sta.variability <- df.tidy.mass %>%
+   select(site_code, block, plot, trt, data) %>%
+   mutate(results = map(data, function(dat) {
+     df <- dat %>%
+       group_by(year_trt) %>%
+       summarise(total.mass = sum(mass)) %>%
+       ungroup() # calculate the total mass at each year
+     output <- df %>%
+       select(year_trt) %>%
```

```
+       rename(end_year = year_trt) %>%
+       filter(end_year >= 4) %>%
+       mutate(VAR.original = map_dbl(end_year, function(ey) {
+         df.sub <- df %>%
+           filter(year_trt >= 1, year_trt <= ey)
+         y <- df.sub$total.mass
+         sd(y) / mean(y)
+       })) %>%
+       mutate(VAR.detrended = map_dbl(end_year, function(ey) {
+         df.sub <- df %>%
+           filter(year_trt >= 1, year_trt <= ey)
+         x <- df.sub$year_trt
+         y <- df.sub$total.mass
+         lmf <- lm(y ~ x)
+         residuals <- lmf$residuals
+         sd(residuals) / mean(y)
+       }))
+   }))
> df.sta.variability %<>%
+   select(-data) %>%
+   unnest(cols = results)
>
> df.sta.variability.treatment <- df.sta.variability %>%
+   filter(trt %in% c("N", "P", "K"))
> df.sta.variability.control <- df.sta.variability %>%
+   left_join(df.controls.selected, .)
>
> df.sta.variability.control %<>%
+   select(-trt, -plot) %>%
+   rename(
+     VAR.original.control = VAR.original,
+     VAR.detrended.control = VAR.detrended
+   )
>
> df.sta.variability.treatment %<>%
+   left_join(df.sta.variability.control)
> df.sta.variability.treatment %<>% ungroup()
> df.sta.variability.treatment %<>% mutate(
```

```
+   temporal.functional.variability = VAR.original / VAR.original.control,
+   temporal.functional.variability.detrended = VAR.detrended / VAR.detrended.control
+ )
>
> # it is possible that some plots of +N, +P, and +K have longer sampling years.
> # In these cases, the code will return a NA value for the stability quantified -
> # using the time range including the year when the control has no data.
> df.sta.temporal.functional.variability.mass <- df.sta.variability.treatment %>%
+   na.omit()
```

2.2 extent of functional change

The extent of functional change was quantified as the mean of the log-response-ratio between the total mass of the treatment plot and the control plot acrossing survey years.

```
> df.tidy.mass.control <- df.tidy.mass %>% left_join(df.controls.selected, .)
> df.tidy.mass.control %<>%
+   select(site_code, block, plot, data) %>%
+   rename(control.plot = plot, control.data = data)
> df.tidy.mass.2 <- df.tidy.mass %>%
+   left_join(df.tidy.mass.control) %>%
+   select(site_code, block, plot, control.plot, trt, data, control.data)
>
> df.mean.lrr <- df.tidy.mass.2 %>%
+   filter(trt %in% c("N", "P", "K")) %>%
+   mutate(results = pmap(list(trt, data, control.data), function(trt.v, dat1, dat2) {
+     dat <- dat1 %>%
+       mutate(g = "experimental") %>%
+       bind_rows(dat2 %>% mutate(g = "control")) %>%
+       select(year_trt, g, category, mass) %>%
+       filter(year_trt %in% intersect(dat1$year_trt, dat2$year_trt))
+     dat.nested <- dat %>%
+       group_by(year_trt, g) %>%
+       summarise(total_mass = sum(mass))
+     lRR.total.mass <- dat.nested %>%
+       pivot_wider(names_from = g, values_from = total_mass) %>%
+       mutate(lrr = log(experimental / control))
+     output <- lRR.total.mass %>%
+       select(year_trt) %>%
```

```
+       rename(end_year = year_trt) %>%
+       filter(end_year >= 4) %>%
+       mutate(mean.lrr = map_dbl(end_year, function(x) {
+         lRR.total.mass %>%
+           filter(year_trt <= x, year_trt >= 1) %>%
+           pull(lrr) %>%
+           abs() %>%
+           mean()
+       }))
+     return(output)
+   }))
>
> df.sta.mean.lrr <- df.mean.lrr %>%
+   select(-data, -control.plot, -control.data) %>%
+   unnest(cols = results)
>
> df.sta.extent.functional.change.mass <- df.sta.mean.lrr %>%
+   rename(extent.functional.change = mean.lrr)
>
> # if the biomass of some plots is zero, the returned stability values will be NA
> df.sta.extent.functional.change.mass <- df.sta.extent.functional.change.mass %>%
+   na.omit()
```

Section 2: Quantify stability components using cover data

This includes the compositional variability (detrended and non-detrended) and compositional resistance, i.e. the extent of compositional change.

2.1 compositional variability

As Turnover.treatment/Turnover.control.

```
> df.tidy.coverage <- df.tidy %>%
+   filter(data.availability != "only biomass data") %>%
+   select(site_code:trt, coverage.data) %>%
+   rename(data = coverage.data)
> df.controls <- df.tidy.coverage %>%
+   unnest(cols=data) %>%
+   ungroup() %>%
```

```
+   filter(trt == "Control") %>%
+   select(site_code, block, plot, year_trt) %>%
+   distinct()
> set.seed(1234)
> df.controls.selected <- df.controls %>% # select the plot with largest year_trt
+   arrange(site_code, block, desc(year_trt)) %>%
+   group_by(site_code, block) %>%
+   slice_head(n = 1) %>%
+   ungroup() %>%
+   select(-year_trt)
> df.sta.compositional.turnover <- df.tidy.coverage %>%
+   mutate(results = map(data, function(dat) {
+     M <- dat %>% # long to wide
+       select(year_trt, Taxon, max_cover) %>%
+       pivot_wider(names_from = Taxon, values_from = max_cover, values_fill = 0)
+     # jaccard distance between community at year_trt
+     dist.m <- vegdist(M %>% select(-year_trt),
+       method = "jaccard",
+       binary = TRUE
+     )
+     dist.m %<>% as.matrix()
+     # only dist between consecutive years
+     dist.v <- diag(dist.m[2:nrow(dist.m), 1:(nrow(dist.m) - 1)])
+     df <- M %>%
+       select(year_trt) %>%
+       filter(year_trt != 0) %>%
+       arrange(year_trt) %>%
+       mutate(dist = dist.v)
+     output <- df %>%
+       select(year_trt) %>%
+       rename(end_year = year_trt) %>%
+       filter(end_year >= 4) %>%
+       mutate(compositional.turnover = map_dbl(end_year, function(ey) {
+         df %>%
+           filter(year_trt <= ey) %>%
+           pull(dist) %>%
+           mean()
+       }))
```

```
+   }))
> df.sta.compositional.turnover %<>%
+   select(-data) %>%
+   unnest(cols = results)
>
> df.sta.compositional.turnover.treatment <- df.sta.compositional.turnover %>%
+   filter(trt %in% c("N", "P", "K"))
> df.sta.compositional.turnover.control <- df.sta.compositional.turnover %>%
+   semi_join(df.controls.selected)
>
> df.sta.compositional.turnover.control %<>%
+   select(-trt, -plot) %>%
+   rename(compositional.turnover.control = compositional.turnover)
>
> df.sta.compositional.turnover.treatment %<>%
+   left_join(df.sta.compositional.turnover.control)
> df.sta.compositional.turnover.treatment %<>%
+   mutate(
+     temporal.compositional.variability =
+       compositional.turnover / compositional.turnover.control
+   )
>
> df.sta.temporal.compositional.variability <- df.sta.compositional.turnover.treatment
> # as the survey range of the control plot is shorter than the treatment plots, -
> # there will be some NA values for the stability quantified using long range data
> df.sta.temporal.compositional.variability <- na.omit(df.sta.temporal.compositional.variability)
```

2.2 extent of compositional change

the mean dissimilarity between the treatment plot and the control plot acrossing survey years; dissimilarity is using Bray-Curtis dissimilarity of log(cover+1)

```
> df.tidy.control <- df.tidy.coverage %>% semi_join(df.controls.selected)
> df.tidy.control %<>%
+   select(site_code, block, plot, data) %>%
+   rename(control.plot = plot, control.data = data)
> df.tidy.coverage.2 <- df.tidy.coverage %>%
+   left_join(df.tidy.control) %>%
+   select(site_code, block, plot, control.plot, trt, data, control.data)
```

```
>
> df.max.mean.bc.numeric <- df.tidy.coverage.2 %>%
+   filter(trt != "Control") %>%
+   mutate(results = pmap(list(trt, data, control.data), function(trt.v, dat1, dat2) {
+     dat <- dat1 %>%
+       mutate(g = "experimental") %>%
+       bind_rows(dat2 %>% mutate(g = "control")) %>%
+       select(year_trt, g, Taxon, max_cover) %>%
+       filter(year_trt %in% intersect(dat1$year_trt, dat2$year_trt))
+     dat.wide <- dat %>%
+       pivot_wider(names_from = "Taxon", values_from = "max_cover", values_fill = 0)
+     dat.nested <- dat.wide %>%
+       group_by(year_trt) %>%
+       nest()
+     dat.nested %<>%
+       mutate(bc.dist = map_dbl(data, function(x) {
+         x <- x %>%
+           select(-g) %>%
+           as.matrix()
+         x <- log(x + 1)
+         out <- x %>%
+           vegdist(., method = "bray", binary = F, diag = TRUE, upper = TRUE) %>%
+           as.matrix()
+         out[1, 2]
+       }))
+     output <- dat.nested %>%
+       select(year_trt) %>%
+       rename(end_year = year_trt) %>%
+       filter(end_year >= 4) %>%
+       mutate(max.bc.numeric = map_dbl(end_year, function(x) {
+         dat.nested %>%
+           filter(year_trt <= x, year_trt >= 1) %>%
+           pull(bc.dist) %>%
+           max()
+       })) %>%
+       mutate(mean.bc.numeric = map_dbl(end_year, function(x) {
+         dat.nested %>%
+           filter(year_trt <= x, year_trt >= 1) %>%
```

```
+           pull(bc.dist) %>%
+           mean()
+       }))
+     return(output)
+   }))
>
> df.sta.max.mean.bc.numeric <- df.max.mean.bc.numeric %>%
+   select(-data, -control.plot, -control.data) %>%
+   unnest(cols = results)
>
> df.sta.extent.compositional.change <- df.sta.max.mean.bc.numeric %>%
+   select(-max.bc.numeric) %>%
+   rename(extent.compositional.change = mean.bc.numeric)
> df.sta.extent.compositional.change <- na.omit(df.sta.extent.compositional.change)
```

Section 3: Compile stability components

```
> df.tidy.stabilities <- df.sta.temporal.functional.variability.mass %>%
+   full_join(df.sta.temporal.compositional.variability) %>%
+   full_join(df.sta.extent.functional.change.mass) %>%
+   full_join(df.sta.extent.compositional.change)
> #only keep the core columns
> df.tidy.stabilities <- df.tidy.stabilities %>%
+   select(site_code, block, trt, plot, end_year, contains("variability"),contains("change"))
```

Section 4: A preliminary view of the stability data

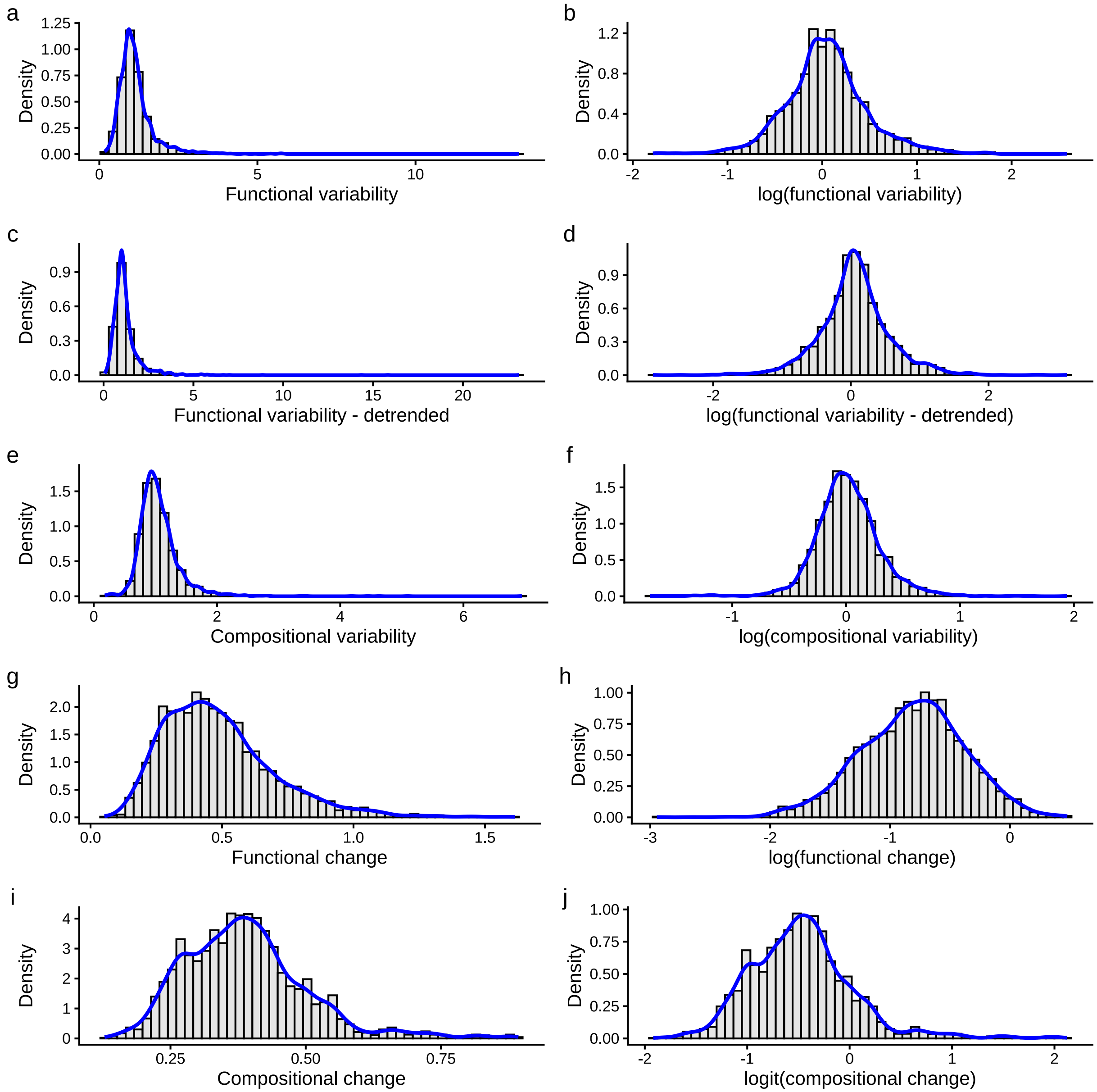
4.1 The frequency distribution of stability components

First have a look at the frequency distribution of the stability components.

```
> logit.f <- function(x) log(x / (1 - x))
> # define a function to plot the frequency distribution of stability components
> ggplot.freq.f <- function(dat, xlab = "x") {
```

```
+   dat %>%
+     set_colnames(value = "x") %>%
+     ggplot(aes(x = x)) +
+     geom_histogram(aes(y = ..density..),
+       color = "black", fill = "gray90", size = 1, bins = 50
+     ) +
+     geom_density(color = "blue", size = 1) +
+     theme_classic() +
+     ylab("Density") +
+     theme(axis.text = element_text(color = "black")) +
+     xlab(xlab)
+ }
>
> p1.1.a <- df.tidy.stabilities %>%
+   select(temporal.functional.variability) %>%
+   ggplot.freq.f(., xlab = "Functional variability")
> p1.1.b <- df.tidy.stabilities %>%
+   select(temporal.functional.variability.detrended) %>%
+   ggplot.freq.f(., xlab = "Functional variability - detrended")
> p1.2 <- df.tidy.stabilities %>%
+   select(temporal.compositional.variability) %>%
+   ggplot.freq.f(., xlab = "Compositional variability")
> p1.3 <- df.tidy.stabilities %>%
+   select(extent.functional.change) %>%
+   ggplot.freq.f(., xlab = "Functional change")
> p1.4 <- df.tidy.stabilities %>%
+   select(extent.compositional.change) %>%
+   ggplot.freq.f(., xlab = "Compositional change")
>
> p2.1.a <- df.tidy.stabilities %>%
+   select(temporal.functional.variability) %>%
+   log() %>%
+   ggplot.freq.f(., xlab = "log(functional variability)")
> p2.1.b <- df.tidy.stabilities %>%
+   select(temporal.functional.variability.detrended) %>%
+   log() %>%
+   ggplot.freq.f(., xlab = "log(functional variability - detrended)")
> p2.2 <- df.tidy.stabilities %>%
```

```
+   select(temporal.compositional.variability) %>%
+   log() %>%
+   ggplot.freq.f(., xlab = "log(compositional variability)")
> p2.3 <- df.tidy.stabilities %>%
+   select(extent.functional.change) %>%
+   log() %>%
+   ggplot.freq.f(., xlab = "log(functional change)")
> p2.4 <- df.tidy.stabilities %>%
+   select(extent.compositional.change) %>%
+   logit.f() %>%
+   ggplot.freq.f(., xlab = "logit(compositional change)")
>
> library(patchwork)
> p.distribution <- (p1.1.a + p2.1.a) / (p1.1.b + p2.1.b) / (p1.2 + p2.2) / (p1.3 + p2.3) / (p1.4 + p2.4)
> p.distribution <- p.distribution +
+   plot_annotation(tag_levels = "a")
> p.distribution
```


a
Density
Functional variability
b
Density
log(functional variability)
c
Density
Functional variability - detrended
d
Density
log(functional variability - detrended)
e
Density
Compositional variability
f
Density
log(compositional variability)
g
Density
Functional change
h
Density
log(functional change)
i
Density
Compositional change
j
Density
logit(compositional change)

Based on the distribution shape above, I transformed the stability components.

```
> df.tidy.stabilities <- df.tidy.stabilities %>%
+   mutate(func_var = log(temporal.functional.variability),
+          func_var_det = log(temporal.functional.variability.detrended),
+          comp_var = log(temporal.compositional.variability),
+          func_change =  log(extent.functional.change),
+          comp_change = logit.f(extent.compositional.change)) %>%
+   select(-contains("variability"), -contains("nal.change"))
> save(df.tidy.stabilities,
+   file = paste0(data.path, "/df.tidy.stabilities.Rdata")
+ )
```

4.2 The correlation between two measure of temporal functional variability

```
> # Fit a linear model
> model <- lm(func_var_det ~ func_var, data = df.tidy.stabilities)
>
> # Extract R-squared and p-value
> summary_model <- summary(model)
> r_squared <- summary_model$r.squared
> p_value <- summary_model$coefficients[2, 4]
>
> # Create the plot
> plot(
+   df.tidy.stabilities$func_var,
+   df.tidy.stabilities$func_var_det,
+   pch = 1, col = "lightgray",
+   xlab = "Functional Variability",
+   ylab = "Detrended Functional Variability",
+ )
>
> # Add regression line
> abline(model, col = "red", lwd = 2)
>
> # Add R-squared and p-value in the upper left corner
> text(
+   x = -1,
+   y = 2.5,
```

```
+   labels = paste0("R² = ", round(r_squared, 3), "\n", "P = ", format.pval(p_value, digits = 3)),
+   adj = c(0, 1),  # Align text to top-left corner
+   cex = 1  # Slightly increase text size
+ )
```